Arc-124M-VL v0.3

Small from-scratch text + image pretrained base model

Project goal

Arc is a compact multimodal base model: text tokens predict the next text token, and image + text prefixes predict caption-style continuations. It is intentionally not an assistant, not an OCR model, and not a document/chart model.

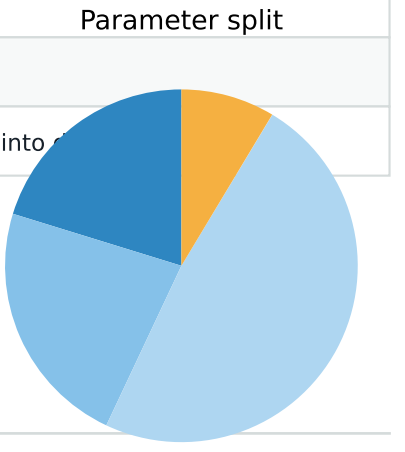
124.4M	113.7M	10.7M	1024
Total params	Text side	Vision side	Context

Text tokens and projected visual tokens both enter the decoder as 768-dimensional vectors.

1. Configuration And Parameter Budget

The exact implemented dimensions from config.json and model.py

Field	Value	Meaning
vocab_size	32768	Number of text tokens
d_model	768	Width of every decoder token vector
n_layers	12	Number of decoder transformer blocks
n_heads	12	Attention heads per decoder block
head_dim	64	Width per attention head
ffn_hidden	2176	SwiGLU/FFN intermediate width
image_size	224	Input image side length
patch_size	16	Vision patch size
vision_resampler_tokens	32	Compressed visual tokens inserted into d

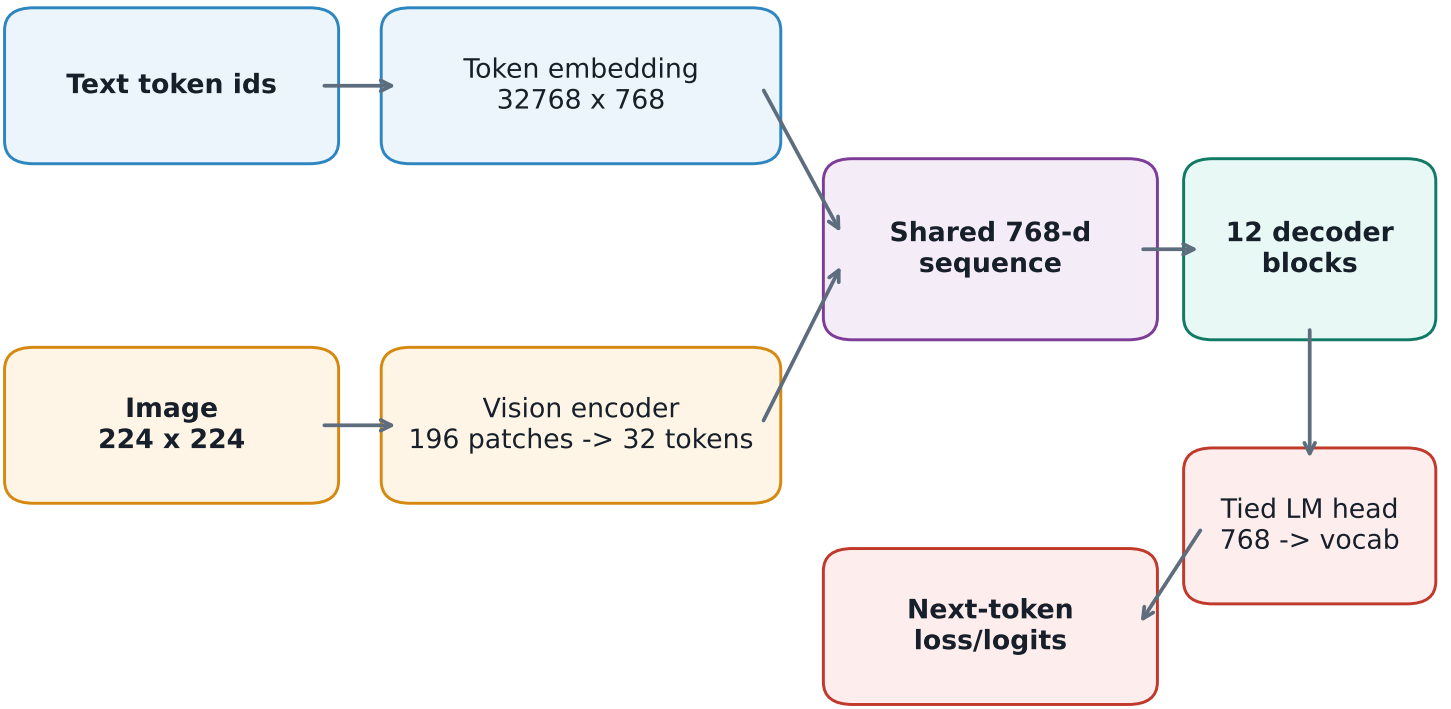


Largest parameter groups

Group	Parameters	Share
decoder FFN	60.16M	48.4%
decoder attention	28.31M	22.8%
token embedding / tied LM head	25.17M	20.2%
vision side	10.71M	8.6%
decoder norms	0.02M	0.0%
final norm	0.00M	0.0%

2. Whole-Model Flow

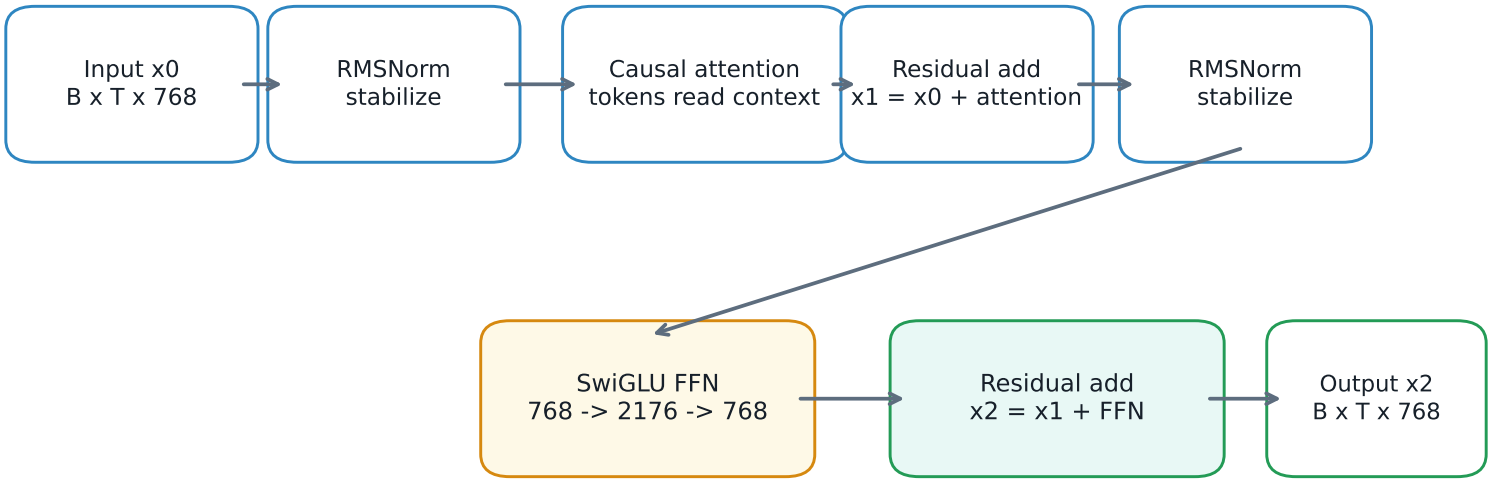
Two input paths, one shared decoder stream



Key idea: image features are not decoded separately. The vision path converts images into 32 soft tokens with the same 768-wide shape as text embeddings. The decoder then attends over one mixed sequence.

3. Decoder Block Deep Dive

The same shape enters and exits every block: B x T x 768

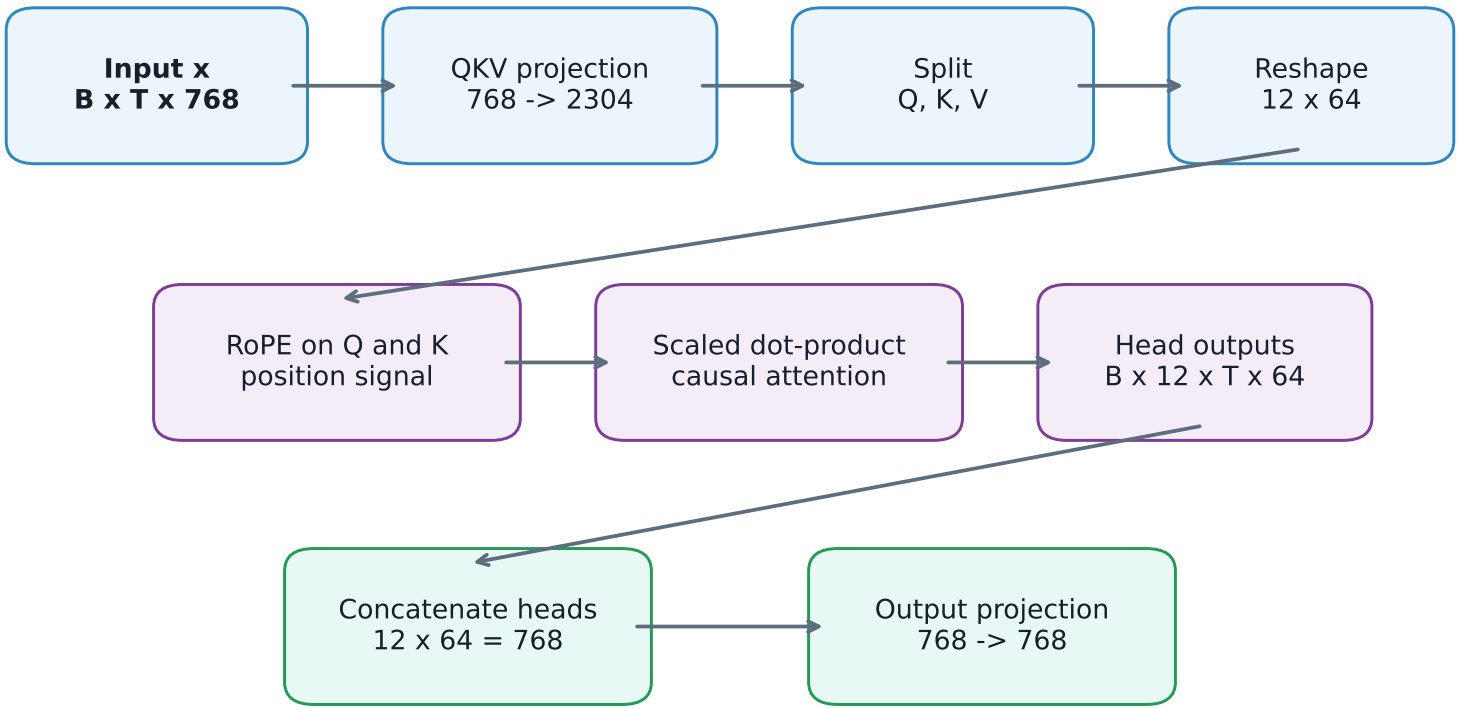



```
x0 = block input
x1 = x0 + attention(RMSNorm(x0))
x2 = x1 + SwiGLU_FFN(RMSNorm(x1))
return x2
```

Attention communicates across tokens. The FFN/MLP processes each token independently after attention has mixed in context. Residual adds preserve the old state and only add learned updates.

4. Attention Mechanics

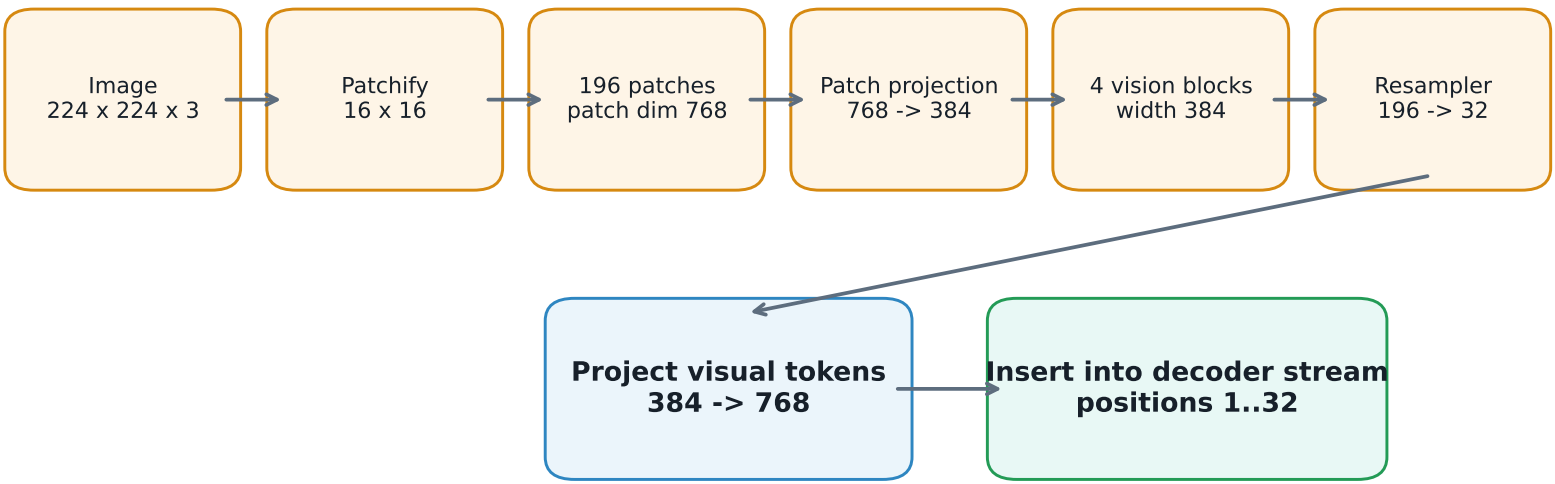
How 768 dimensions become 12 heads and then merge again



The split is safe because 768 divides cleanly by 12. The model first learns Q/K/V projections, so heads are not just raw fixed slices of the original embedding. Each head sees a learned view, then the output projection mixes the heads back into one 768-wide token state.

5. Vision Path

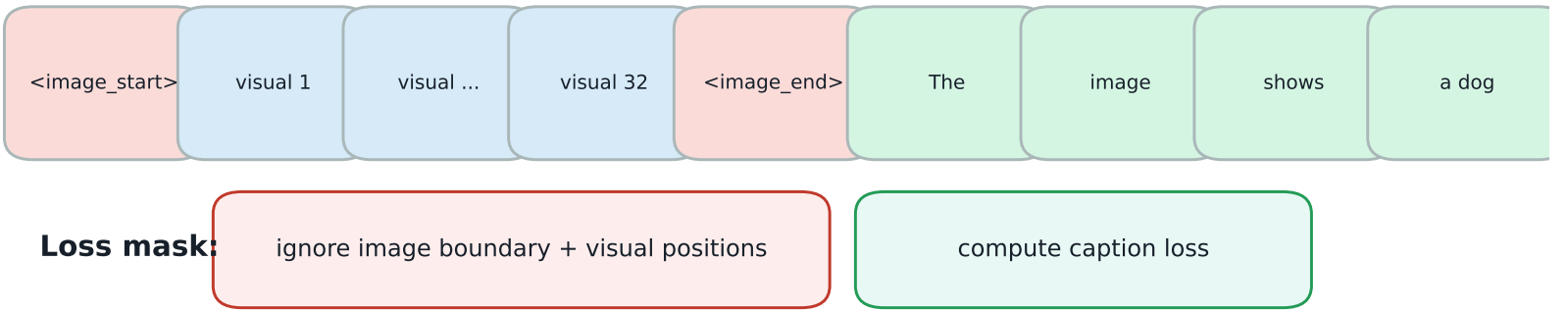
A small native image front-end produces decoder-compatible visual tokens



The vision encoder is intentionally small: it learns basic object, color, scene, and action grounding. It is not designed for OCR, documents, charts, or fine-grained visual reasoning. The resampler keeps image cost low by giving the decoder 32 visual tokens instead of all 196 patch tokens.

6. Multimodal Sequence And Loss Mask

The image is a prefix; loss is only on caption/text tokens



The visual tokens are embeddings inserted into the sequence, not normal vocabulary words. The model is trained to predict the caption continuation, so the loss ignores the visual prefix and starts on the natural-language text.

Caption prefixes are varied during Stage 2: empty prefix, 'A photo of', 'The image shows', 'In this scene,', and 'This is'. This teaches image + text prefix -> continuation rather than only image -> caption.

7. Training System

Stage 1 builds language; Stage 2 teaches image grounding

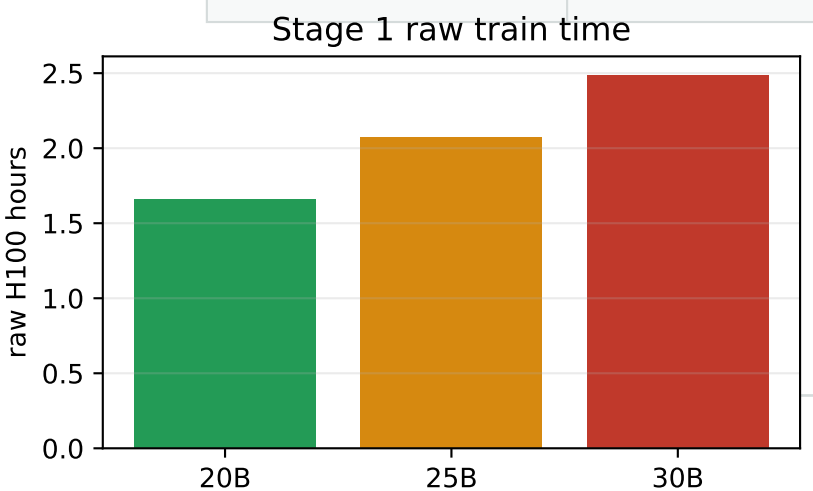
Phase	Trainable parts	Purpose
Stage 1	text decoder only; vision frozen	learn language continuation
Stage 2a	vision encoder, resampler, projector, image boundary rows	align images to decoder space
Stage 2b	Stage 2a + final 2 decoder blocks	learn image/text fusion with low LM LR
Stage 2c	optional full low-LR unfreeze	only if validation remains stable

The essential validation is correct-image vs wrong-image vs blank-image caption loss. If correct image loss is not lower, the decoder is ignoring visual tokens.

8. Data And Compute Plan

The current practical execution plan

Area	Plan
Text data	60% FineWeb-Edu, 40% DCLM 100BT shuffled
Text target	20B safe, 25B stretch; 30B only if budget allows
Vision data	COCO Captions + AnyModal/Flickr30k
Stage 1 hardware	8x H100 SXM, bf16, torch.compile, batch 32/GPU
Measured H100 speed	about 3.35M tokens/sec after compile warmup
Stage 2 hardware	1x or 2x RTX 4090 should be enough for v0



Budget note

Data sharding is slow but cheap and should happen on a low-cost pod attached to the same network volume. H100 time should be reserved for the actual Stage 1 train.

9. Code Map For Reviewers

Files to inspect for architecture, training, and data flow

File	Why it matters
src/arc/model.py	Complete neural architecture: decoder, vision encoder, resampler, forward pass.
config.json + src/arc/config.py	Actual dimensions and model settings.
src/arc/data_vision.py	Image-caption sequence construction and loss mask.
src/arc/train_text.py	Stage 1 DDP text pretraining loop.
src/arc/train_vl.py	Stage 2 multimodal training loop and validation.
src/arc/training_plan.py	Stage 2 freeze/unfreeze policy.
src/arc/tokenizer.py	Byte BPE tokenizer and image special tokens.
configs/*.jsonl	Dataset mixture definitions.

Minimal architecture bundle: config.json, src/arc/model.py, src/arc/config.py. Full systems bundle: add data_vision.py, train_text.py, train_vl.py, training_plan.py, tokenizer.py, README.md, and configs.